

JavaScript Key Learnings — Question 5

OJT Assessment Prep — Constructor Functions, Prototype, setTimeout & Closures

1. Constructor Functions — The Old-School Way

Before ES6 classes, JS used **constructor functions** to create objects. Called with **new**, just like classes.

```
function Person(name) { // Capital letter = convention for constructors
  this.name = name;      // this = the new object being created
  this.age = 0;
}

const p = new Person("John");
p.name // → "John"
```

Feature	Constructor Function	ES6 Class (Q3)
Define object	function Foo() { this.x = 1 }	class Foo { constructor() { this.x = 1 } }
Add methods	Foo.prototype.bar = function(){}	bar() {} inside class body
Create object	new Foo()	new Foo()
Era	ES5 — legacy code	ES6 — modern code

2. prototype — Adding Shared Methods

With constructor functions, methods go on **.prototype** — not inside the function. All objects share one copy, saving memory.

■ **Wrong — method inside constructor (new copy for every object):**

```
function Person(name) {
  this.name = name;
  this.greet = function() { ... } // new copy created every time!
}
```

■ **Correct — method on prototype (one shared copy):**

```
function Person(name) {
  this.name = name; // properties go inside
}

Person.prototype.greet = function() { // methods go on prototype
  console.log("Hello " + this.name);
};

const p = new Person("John");
p.greet(); // → "Hello John"
```

■ **Rule:** properties (data) go inside constructor, methods go on prototype.

3. setTimeout — Delayed Execution

setTimeout runs a function after a specified delay in milliseconds. Code keeps running while it waits — it is non-blocking.

```
setTimeout(function() {
  console.log("runs after 3 seconds");
}, 3000);
// ↑ 3000ms = 3 seconds

// 1000ms = 1s, 2000ms = 2s, 3000ms = 3s
```

4. Context Loss in setTimeout

Same problem as Q3 (addEventListener) — but now inside **setTimeout**. The callback loses the constructor's **this**.

■ **Broken — this is lost inside setTimeout callback:**

```
ToastNotification.prototype.show = function() {
  setTimeout(function() {
    this.div.remove(); // ■ this = window, not the instance!
  }, 3000);
};
```

5a. Fix with Closure — Capturing this in a Variable

A **closure** means a function remembers variables from its outer scope. Capture **this** before entering **setTimeout**:

■ **Fix with closure:**

```
ToastNotification.prototype.show = function() {
  let self = this; // capture this in outer scope
  setTimeout(function() {
    self.div.remove(); // ■ self never loses reference
  }, 3000);
};
```

■ **self (or that)** is just a regular variable — regular variables don't lose context like **this** does.

5b. Fix with .bind(this) — Alternative Approach

Same fix as Q3 — lock **this** permanently with **.bind()**:

■ **Fix with .bind(this):**

```
ToastNotification.prototype.show = function() {
  setTimeout(function() {
    this.div.remove(); // ■ this = instance, locked by bind
  }.bind(this), 3000); // ← bind goes before the closing ), 3000
};
```

6. Removing DOM Elements — .remove()

An element can remove itself from the DOM by calling **.remove()** directly on itself:

```
const div = document.createElement("div");
document.body.appendChild(div);

// Later — remove it
div.remove(); // removes itself from the page

// ■ Wrong way:
document.removeChild(div); // removeChild needs a parent element, not document
```

7. Always Store Elements on this (Recap from Q3)

This bug appeared again in Q5 — a good reminder:

■ **Wrong — local variable, dies after constructor:**

```
function ToastNotification(message) {
  let div = document.createElement("div"); // local — gone after constructor
} // show() cannot access div!
```

■ **Correct — stored on this, accessible everywhere:**

```
function ToastNotification(message) {
  this.div = document.createElement("div"); // lives on the object
} // show() can use this.div or self.div anytime
```

■■ Final Correct Code

```
function ToastNotification(message) {
  this.message = message;           // store message
  this.div = document.createElement("div"); // store element on this
  this.div.textContent = this.message; // set content
}

ToastNotification.prototype.show = function() {
  let self = this;                  // capture context
  document.body.append(self.div);   // show the toast
  setTimeout(function() {
    self.div.remove();              // self-dismiss after 3s
  }, 3000);
};

let a = new ToastNotification("Action Successful");
a.show();
```

■ Q3 vs Q5 — Same Problem, Different Location

	Question 3	Question 5
Syntax	ES6 class	Constructor function
Methods	Inside class body	On .prototype

Context loss in	<code>addEventListener</code>	<code>setTimeout</code>
Fix	<code>.bind(this)</code>	closure (self) or <code>.bind(this)</code>

Made during OJT JS prep session — June 2026